Rechnerorganisation 2

CPU Documentation

Group 2

This document describes the design, implementation, and verification of our custom CPU developed as part of the RO2 course project. The goal was to collaboratively design a simple, yet functional CPU architecture, implement it in VHDL, and verify its operation through simulation.

Name	Matrikelnummer	Email
Berke Özmen	3812619	st196187@stud.uni-stuttgart.de
Dominik Knaup	3721438	st187468@stud.uni-stuttgart.de
Lukas Kurz	3655410	st182338@stud.uni-stuttgart.de
Maximilian Rau	3731389	st188391@stud.uni-stuttgart.de
Mouhammad Mahdi Bazzoun	3743872	st189618@stud.uni-stuttgart.de
Muhammed Sahin	3475535	st169152@stud.uni-stuttgart.de

1. Architecture Overview

The CPU (`cpu.vhd`) is a PicoBlaze-compatible 8-bit processor with a 2-stage pipeline (fetch/decode and execute/writeback). It connects five main components:

- **Control Unit** – decodes the 18-bit instruction word from ROM into all datapath, memory, and IO control signals. Contains the program counter and branch logic.
- **Datapath** – register file (16 × 8-bit) and ALU with pipeline registers that delay control signals by one cycle to align them with the execute stage.
- **Hazard Detector** – combinational logic that compares the execute-stage write address against the decode-stage read addresses. Stalls the pipeline for one cycle on RAW conflicts.
- **Memory Unit** – instruction ROM (4096 × 18-bit, asynchronous read) and scratchpad RAM (256 × 8-bit, synchronous write, asynchronous read).
- **IO Unit** – 256-port input/output interface. Outputs are registered, inputs are combinational.

1.1. Signal Flow

Each cycle, the ROM delivers an instruction at the current PC. The control unit decodes it immediately, producing ALU operation, register addresses, immediate values, and branch conditions. These signals enter the datapath's pipeline registers and take effect one cycle later in the execute stage.

IO and memory control signals (`io_rd`, `io_wr`, `mem_read`, `mem_write`) are delayed by one cycle in `cpu.vhd` so they fire during execute, when the datapath has valid address and data outputs. On a stall, flush, or reset these delayed signals are cleared to prevent unwanted writes.

The data-input mux in `cpu.vhd` selects the source for register writeback: IO data (INPUT), RAM data (FETCH), or zero (default, letting the ALU result through via the datapath).

1.2. Top-Level Interface

Port	Direction	Width	Description
`clk`	in	1	System clock
`reset`	in	1	Synchronous reset
`in_ports`	in	256×8	External input ports
`out_ports`	out	256×8	External output ports

2. Component Descriptions

2.1. Control Unit

The **Control Unit** decodes the 18-bit instruction word and generates all control signals that steer the datapath, memory, IO, and program flow. It is split into five files with clearly separated responsibilities: `controlunit_pkg.vhd` (shared types), `instruction_decoder.vhd` (opcode decoding), `branch_logic.vhd` (jump evaluation), `program_counter.vhd` (PC register), and `control_unit.vhd` (integration wrapper).

2.1.1. Instruction Decoder

The Instruction Decoder (`instruction_decoder.vhd`, `controlunit_pkg.vhd`) is a purely combinational block. It takes the 18-bit instruction word and produces all control signals in a single combinational pass (no clock needed).

2.1.1.1. Instruction Format

Every instruction is 18 bits wide and follows a fixed layout:

Bits	Field	Description
`17..12`	`opcode`	6-bit operation code
`11..8`	`sX`	Destination / source register A (s0–sF)
`7..4`	`sY`	Source register B (register-addressed instructions)
`7..0`	`kk` / `pp`	8-bit immediate or port address (immediate instructions)
`11..0`	`aaa`	12-bit jump target (jump instructions)

The opcode is decoded with a `case` statement. Each branch sets only the signals that differ from the safe defaults (all writes disabled, `ALU_NOP`, no jump).

2.1.1.2. Generated Control Signals

Signal	Purpose
`alu_op`	ALU operation (`ALU_ADD`, `ALU_SUB`, `ALU_AND`, …, `ALU_TEST`, `ALU_COMPARECY`, `ALU_TESTCY`, `ALU_PASS_B`)
`alu_src`	ALU B-input source: `0` = register sY, `1` = immediate `kk`
`imm8`	8-bit immediate value (valid when `alu_src = '1'`)
`reg_dst`	Destination register index (sX)
`reg_src_b`	Source register index B (sY)
`reg_write`	Enables register writeback
`flag_write`	Enables carry/zero flag update (only for flag-affecting instructions)
`mem_read`	Scratch-pad read (FETCH)
`mem_write`	Scratch-pad write (STORE)
`io_rd`	IO read (INPUT)
`io_wr`	IO write (OUTPUT)
`jump_cond`	Branch condition (`J_NONE`, `J_Z`, `J_NZ`, `J_C`, `J_NC`, `J_UNCOND`)
`reads_reg_a`	Instruction reads register A – used by Hazard Detector
`reads_reg_b`	Instruction reads register B – used by Hazard Detector

2.1.1.3. Instruction Groups

Arithmetic / Logic (`ADD`, `ADDCY`, `SUB`, `SUBCY`, `AND`, `OR`, `XOR`): Each exists in a register variant (`sX, sY`) and an immediate variant (`sX, kk`). All set `reg_write = '1'` and `flag_write = '1'`. Immediate variants set `alu_src = '1'` and `reads_reg_b = '0'`.

Flag-only operations:

- `COMPARE` reuses `ALU_SUB` but with `reg_write = '0'` – only updates flags.
- `TEST` uses `ALU_TEST` with `reg_write = '0'` – computes `sX AND sY/kk` for the Z flag, and sets C to the odd parity of the AND result.

- `COMPARECY` uses `ALU_COMPARECY` with `reg_write = '0'` – like `SUBCY` but flags-only. Chains the zero flag via `z_i`: `Z = (result = 0) AND z_i`, enabling multi-byte comparisons.
- `TESTCY` uses `ALU_TESTCY` with `reg_write = '0'` – like `TEST` but chains both carry and zero: `C = parity(result) XOR c_i`, `Z = (result = 0) AND z_i`.

Data transfer (`LOAD`): Uses `ALU_PASS_B` to pass the source (register or immediate) through the ALU to the destination register. Sets `reads_reg_a = '0'` because sX is only written, not read – this avoids unnecessary pipeline stalls.

IO instructions (`INPUT`, `OUTPUT`): The datapath's `address_out` is wired directly to the ALU B-input (`B_alu`), so the port address (register sY or immediate `kk`) reaches `address_out` without going through the ALU computation.

Scratch-pad memory (`FETCH`, `STORE`): Same addressing mechanism as IO – `address_out` comes directly from `B_alu`, so the memory address (register value or immediate) bypasses the ALU result path.

Shifts and Rotates (`RL`, `RR`, `SL0`, `SL1`, `SLA`, `SLX`, `SR0`, `SR1`, `SRA`, `SRX`): Single-operand instructions that only use register A. Each sets `reads_reg_b = '0'`, `reg_write = '1'`, and `flag_write = '1'`. The carry flag is set to the bit shifted out (bit 7 for left shifts, bit 0 for right shifts). `SRX` performs arithmetic right shift (sign extension: bit 7 is preserved). `SRA` shifts right and inserts the carry flag.

Jumps (`JUMP`, `JUMP Z`, `JUMP NZ`, `JUMP C`, `JUMP NC`): Set `jump_cond` to the corresponding condition. No register reads (`reads_reg_a = '0'`, `reads_reg_b = '0'`), no writes.

2.1.2. Branch Logic, Program Counter & Integration

2.1.2.1. Branch Logic (`branch_logic.vhd`)

The Branch Logic is a purely combinational block that decides whether a jump should be taken. It receives `jump_cond` and `jump_enable` from the decoder/control unit, together with the current flags (`carry_in`, `zero_in`), and produces `branch_taken`. The `jump_enable` signal is `'1'` for any jump instruction (`J_UNCOND`, `J_Z`, `J_NZ`, `J_C`, `J_NC`) and `'0'` for `J_NONE`. When `jump_enable = '0'`, `branch_taken` is always `'0'` regardless of flag values:

Condition	Taken when
`J_UNCOND`	Always
`J_Z`	`zero_in = '1'`
`J_NZ`	`zero_in = '0'`
`J_C`	`carry_in = '1'`
`J_NC`	`carry_in = '0'`
`J_NONE`	Never (not a jump instruction)

The flags `carry_in` and `zero_in` come from the datapath with *forwarding*: if the instruction currently in the execute stage updates flags (`flag_write = '1'`), the fresh ALU output is forwarded immediately. Otherwise the registered flag values from a previous instruction are used. This ensures that a conditional jump directly after a flag-affecting instruction (e.g. `COMPARE` followed by `JUMP Z`) evaluates the correct flags without needing an extra wait cycle.

2.1.2.2. Program Counter (`program_counter.vhd`)

The Program Counter is a 12-bit synchronous counter with three behaviors:

- **Reset** (`reset = '1'`): `pc_reg` is cleared to `0x000`.

- **Branch/Jump** (`enable = '1'` and `branch_jump = '1'`): `pc_reg` loads the absolute target address `jump_address` (taken from `instruction(11..0)`).
- **Sequential** (`enable = '1'` and `branch_jump = '0'`): `pc_reg` increments by 1.
- **Stall** (`enable = '0'`): `pc_reg` holds its value – the same instruction is re-fetched.

Pipeline stages, hazard detection, stall and flush behavior are described in the Pipeline section.

2.1.2.3. Verification (Simulation)

Verification for the Control Unit is split into focused testbenches under `sim/ControlUnit/`:

- `instruction_decoder_tb.vhd`: combinational decoding of opcodes into control signals.
- `branch_logic_tb.vhd`: jump-condition evaluation (`J_Z`, `J_NZ`, `J_C`, `J_NC`, `J_UNCOND`) independent of the PC.
- `program_counter_tb.vhd`: synchronous reset, enable/hold behavior, increment, and absolute jumps.
- `control_unit_tb.vhd`: integration checks (stall gating, PC freeze/advance, and `flush_out` behavior on taken jumps).

2.1.3. Figures

The following figures describes the architecture of the Control Unit

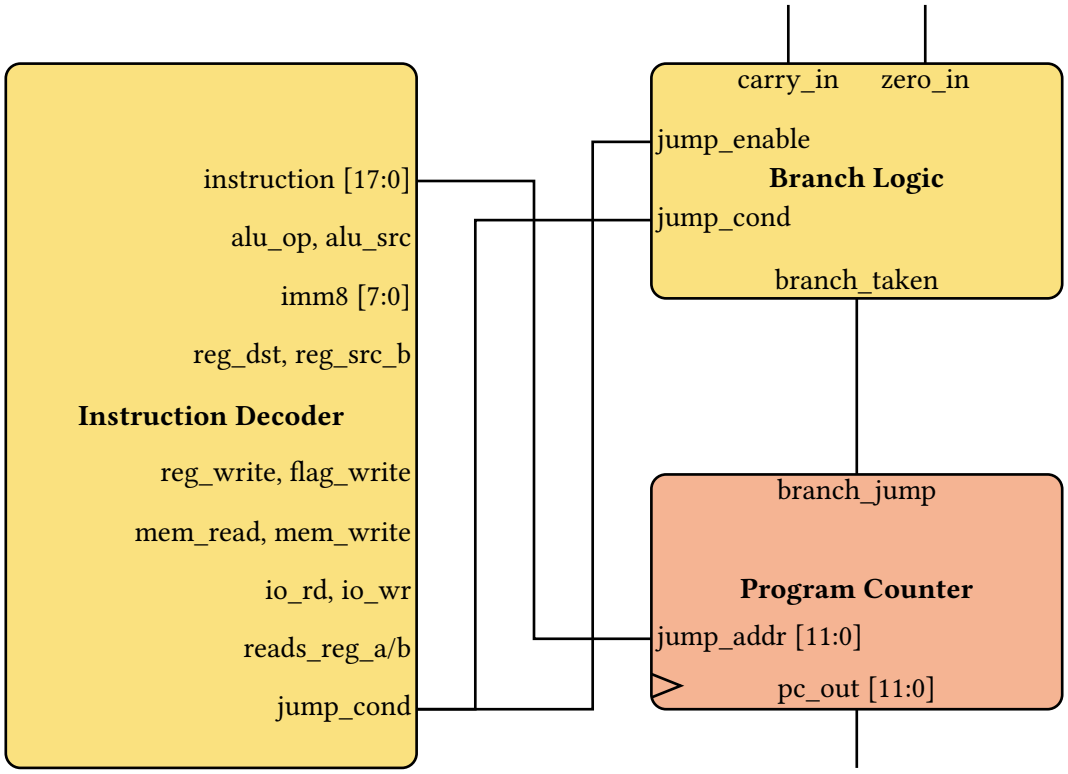


Figure 1: Block diagram: Control Unit – Instruction Decoder, Branch Logic, and Program Counter.

Figure description: This block diagram shows the internal partitioning of the Control Unit. The `instruction` bus feeds the Instruction Decoder, which produces the datapath control (`alu_op`, `alu_src`, register indices, write enables) and the jump condition `jump_cond`. `jump_cond` together with the ALU flags `Z`/`C` is evaluated by the Branch Logic. The resulting jump decision controls whether the Program Counter increments normally or loads the target address.

2.2. Data Path

The datapath connects the register file with the ALU and handles writeback operations. It executes arithmetic, logic, shift and rotate operations on data stored in registers.

The datapath consists of two main components: (1) the register file, which holds the 16 general-purpose 8-bit registers (s0 to sF), and (2) the ALU, which performs the actual computation on the register values.

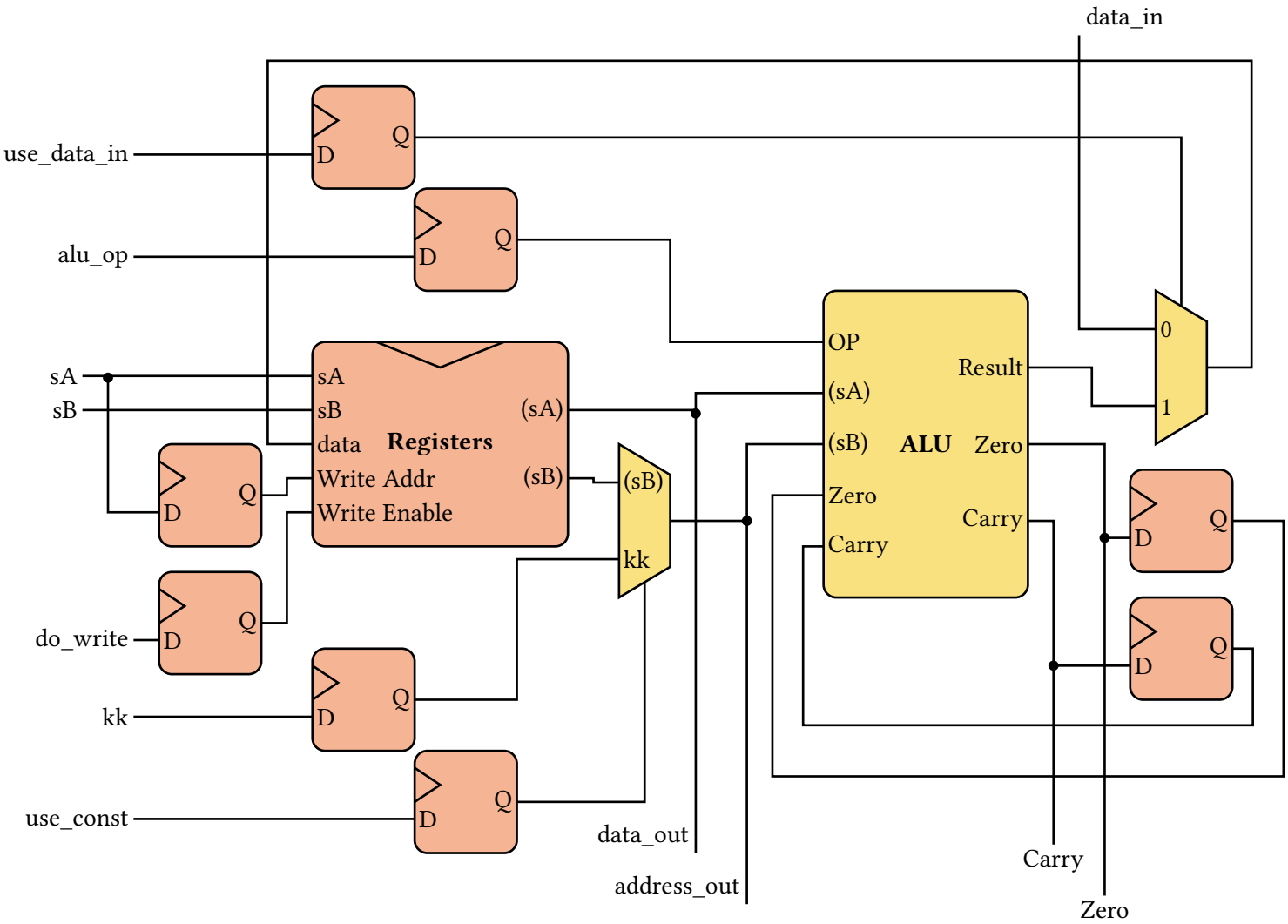


Figure 2: Simplified Circuit Diagram of the Datapath.
Not shown are clock, reset, stall and flush signals as well as the multiplexer controlling whether flags are updated.

The register file provides two read ports and one write port. Both read ports can be accessed simultaneously, allowing the ALU to receive two operands in the same clock cycle.

As this processor only implements instructions of the one and two-address format the first operand (sX) will overwritten by the result.

Read operations are registered, meaning the output appears one clock cycle after the address is provided.

2.2.1. ALU

The ALU receives two 8-bit operands and produces an 8-bit result along with carry and zero flags. The second operand can come either from the register file or from an immediate constant, controlled by the `use_const_in` signal.

The full list of ALU operations is in the Appendix.

The COMPARE instruction uses `ALU_SUB` with register write disabled (flags-only). The TEST instruction uses `ALU_TEST` with register write disabled (flags-only): the Z flag reflects whether `(sX AND sY/kk)` is zero, and the carry flag is set to the odd parity of the AND result. COMPARECY and TESTCY are the carry-chaining variants: they use `ALU_COMPARECY` and `ALU_TESTCY` respectively, also flags-only. Both propagate the zero flag via `z_i` so that multi-byte comparisons and tests work correctly.

The datapath has several ways to move data without performing arithmetic:

Mode	Data Flow	Used For
`ALU_NOP`	Outputs zero	Pipeline stalls (bubble insertion)
`ALU_PASS_B`	Outputs B (register or constant)	LOAD instructions: copies sY to sX, or loads constant kk into sX
`use_data_in`	Bypasses ALU, writes external data directly	INPUT and FETCH instructions: data from IO or memory goes straight to register

`LOAD sX, kk` uses `ALU_PASS_B` with `use_const_in = 1`, so the constant is fed into the B input and passed through. The `use_data_in` path is different: it bypasses the ALU result entirely and selects `data_in` for write-back instead.

2.2.2. Flags

The datapath maintains two status flags:

Flag	Width	Description
Carry	1 bit	Set when an arithmetic operation produces a carry or borrow
Zero	1 bit	Set when the result of an operation is zero

Flags are only updated when the `flag_write_in` signal is active, so instructions like LOAD do not affect the flags.

The flag outputs use a bypass path to provide new flag values immediately when a flag-affecting instruction is in the execute stage. Without this, branch instructions (like `JUMP.Z`) would not see the result of the previous instruction in time.

Pseudocode:

```
carry_flag_out = carry_s          when flag_write_delay1 = 1
                 carry_reg        otherwise

zero_flag_out  = 1                when flag_write_delay1 = 1 AND writeback = 0x00
                 0                when flag_write_delay1 = 1 AND writeback != 0x00
                 zero_reg         otherwise
```

When `flag_write_delay1` is active (the pipelined version, aligned with the execute stage), the combinational output from the ALU (`carry_s`) and the writeback comparison are used directly. Otherwise, the registered values (`carry_reg`, `zero_reg`) are output. So a sequence like:

Assembly:

```
SUB s0, s1      ; sets zero flag if s0 - s1 = 0
JUMP.Z label    ; can branch based on the SUB result
```

works without needing an extra cycle.

2.2.3. Writeback

The write-back mux selects what data gets written to the register file. There are two sources:

Source	Selected when	Used by
ALU result	`use_data_in = 0`	Most instructions (ADD, SUB, LOAD, etc.)
External data	`use_data_in = 1`	INPUT and FETCH instructions

Pseudocode:

```
writeback_s <= data_in when use_data_delay1_s = '1' else alu_result;
```

For INPUT, data comes from the IO unit. For FETCH, data comes from data memory. The ALU result is ignored and external data goes straight to the destination register.

2.2.4. Interface

Signals exposed by the datapath:

Signal	Direction	Width	Description
`clock_in`	Input	1	System clock
`reset_in`	Input	1	Synchronous reset
`stall_in`	Input	1	Pipeline stall signal
`flush_in`	Input	1	Pipeline flush signal (clears execute stage on branch)
`op_in`	Input	-	ALU operation to perform
`addrA_in`	Input	4	Register address for operand A / write destination
`addrB_in`	Input	4	Register address for operand B
`const_in`	Input	8	Immediate constant value
`use_const_in`	Input	1	Select constant instead of register B
`data_in`	Input	8	External data input (from memory or IO)
`use_data_in`	Input	1	Select external data for write-back
`do_write_in`	Input	1	Enable register write
`flag_write_in`	Input	1	Enable flag update
`address_out`	Output	8	ALU B-input (sY or kk) used as address (for OUTPUT/STORE/INPUT/FETCH)
`data_out`	Output	8	Value of register A (for OUTPUT/STORE)
`carry_flag_out`	Output	1	Current carry flag value
`zero_flag_out`	Output	1	Current zero flag value

Signal	Direction	Width	Description
`write_addr_out`	Output	4	Delayed write address (for hazard detection)
`write_enable_out`	Output	1	Delayed write enable (for hazard detection)

2.3. Memory Unit

The memory unit follows a Harvard architecture, with separate memories for instructions and data. The instruction memory is 18 bits wide to fit the full instruction word, while the data memory uses 8-bit words matching the ALU and register file.

2.3.1. Instruction Memory (ROM)

The ROM stores the program as a constant array. Reads are asynchronous, so the instruction is available in the same cycle the program counter updates.

Helper functions (`Op_Load_Im`, `Op_Add_Reg`, etc.) encode instructions from human-readable parameters into 18-bit machine words, replacing raw binary strings.

The `distributed` `rom_style` attribute tells the synthesizer to use LUTs instead of block RAM.

Parameter	Value	Description
Data Width	18 bits	Opcode + register addresses + immediate value
Address Width	12 bits	4096-word address space for jump targets
Read Mode	Asynchronous	Instruction available without clock delay

2.3.2. Data Memory (RAM)

The data memory is an 8-bit scratchpad. Reads are asynchronous (result available in the same cycle), writes are synchronous to the rising clock edge.

Writing to address 0xFF prints the written value to the simulation console, which is useful for debugging without a waveform viewer.

Parameter	Value	Description
Data Width	8 bits	Matches ALU word size
Address Width	8 bits	256 locations
Write Mode	Synchronous	Registered on rising clock edge
Read Mode	Asynchronous	Single-cycle read

2.3.3. Verification

The `MemoryUnit_TB` tests basic read/write, write protection (verifying that data is not written when `Write_En = 0`), the debug port at 0xFF, and a sweep over a range of addresses.

The `cpu_stress_tb` runs the Fibonacci program on the full CPU and checks:

- Correct output sequence (1, 1, 2, 3, 5)
- Recovery after a mid-execution reset
- No undefined outputs after input port noise

2.4. IO Unit

The IO unit connects the processor to external ports. It translates INPUT and OUTPUT instructions into read and write operations within a fixed 256-port address space, keeping the core independent of device details. Inputs are combinational signals, while outputs are stored in clocked registers.

It uses the io_types_pkg.port_array type (256×8-bit vectors) for in_ports and out_ports, allowing direct connection of external devices without modifying the unit.

2.4.1. Interface

Port	Direction	Width	Description
`clk`	in	1	System clock
`rst`	in	1	Synchronous reset
`io_rd`	in	1	Read enable (active high)
`io_wr`	in	1	Write enable (active high)
`port_pp`	in	8	Port address (0–255)
`cpu_data_in`	in	8	Data from datapath for OUTPUT
`io_data_out`	out	8	Data to datapath for INPUT
`in_ports`	in	256×8	External input port values
`out_ports`	out	256×8	Registered output port values

2.4.2. Internal Architecture

The IO Unit consists of two independent paths:

- Output path: a register array (`out_ports_reg`) storing the current values of all output ports.
- Input path: a combinational multiplexer selecting one input port based on `port_pp`.

2.4.3. Output Operation

```
process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            out_ports_reg <= (others => (others => '0'));
        elsif io_wr = '1' then
            out_ports_reg(to_integer(unsigned(port_pp))) <= cpu_data_in;
        end if;
    end if;
end process;

out_ports <= out_ports_reg;
```

On every rising clock edge:

- If rst is asserted, all output registers are cleared.
- If io_wr is active, the value on cpu_data_in is written into the addressed port register, while other ports retain their previous values.

The external out_ports bus is continuously driven by out_ports_reg, so each output port reflects the last written value immediately after the clock edge.

2.4.3.1. Input Operation

```
io_data_out <= in_ports(to_integer(unsigned(port_pp))) when io_rd = '1'
  else (others => '0');
```

The input path is purely combinational:

- When io_rd is asserted, the value of the selected input port appears on io_data_out without any clock delay.
- When io_rd is deasserted, io_data_out is driven to zero to prevent stale data from propagating into the datapath.

2.4.4. Reset Behavior

A synchronous reset (rst) clears all output registers to x“00”. The input path is unaffected by reset, as it only responds to io_rd. This guarantees a defined initial state for all external connections.

2.5. Pipeline and Hazard Handling

The CPU uses a 2-stage pipeline. Each instruction takes 2 clock cycles to complete, but under normal operation a new instruction starts every cycle.

2.5.1. Pipeline Stages

Stage	Cycle	Operations
Fetch/Decode	1	Instruction is read from ROM. Control unit decodes opcode and operands. Register file reads source registers (A and B).
Execute/Writeback	2	ALU performs the operation. Result is written back to the destination register.

The pipeline registers in the datapath delay control signals (ALU operation, write enable, flag write, etc.) by one cycle so they stay aligned with the data flowing through the register file and ALU.

2.5.2. Data Hazards

A RAW (Read After Write) hazard occurs when an instruction in the decode stage needs to read a register that the instruction in the execute stage is about to write. Since the write hasn’t happened yet, the decode stage would read stale data.

Example:

```
LOAD s0, 0x42    ; Execute stage: writing 0x42 to s0
ADD  s1, s0      ; Decode stage: wants to read s0, but s0 still has old value
```

2.5.3. Hazard Detection

The CPU detects hazards by comparing the write address of the instruction in execute with the read addresses of the instruction in decode:

Pseudocode:

```
stall = write_enable_out
    AND ((write_addr = addrA AND reads_reg_a = 1)
      OR (write_addr = addrB AND reads_reg_b = 1))
```

The `reads_reg_a` and `reads_reg_b` signals come from the decoder and indicate whether the current instruction actually reads each register operand. This avoids false stalls for instructions that don't read a given register (LOAD and FETCH don't read sX, immediate-form instructions don't read sY, jumps don't read either).

2.5.4. Stall Behavior

When a hazard is detected, the CPU stalls for one cycle:

Action	Effect
PC held	Same instruction stays in decode stage
Bubble inserted	Execute stage gets neutralized control signals

During a stall or flush, the datapath clears all execute-stage control signals:

Signal	Value during stall/flush
`op_alu_s`	ALU_NOP
`do_write_delay1_s`	0 (no register write)
`flag_write_delay1_s`	0 (flags unchanged)

After the stall, the execute stage completes its write and the decode stage reads the correct value.

2.5.5. Flush Behavior

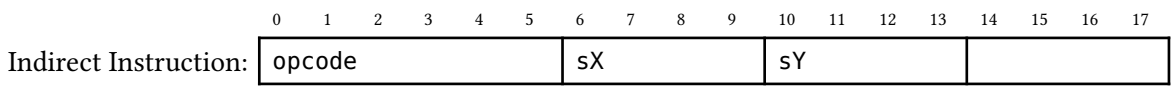
Because fetch and decode happen in the same stage, no instruction after a jump has been fetched when the jump is decoded. A taken jump loads the target into the PC, so the next instruction in decode is already the target and there is no delay slot. The `flush_in` signal clears the execute-stage pipeline registers just like a stall (inserting a bubble), but unlike a stall the PC is not held. The bubble replaces the jump itself, which has no execute-stage effects anyway, so the flush is a safety measure rather than a correctness requirement.

3. Instruction Set

In this section, we describe the available instructions of a simple RISC processor inspired by PicoBlaze.

3.1. Instruction layout

Field	Size	Description
opcode	6 bits	operation to be performed
sX	4 bits	destination and first source register
sY	4 bits	second source register
kk	8 bits	immediate 8-bit constant value
aaa	12 bits	jump target address



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Immediate Instruction:	opcode						sX				kk							
Jump Instruction:	opcode						aaa											

3.2. Available instructions

Instruction	Opcode	Description
nop	000000	//
add	000010	sX += sY
add.im	000011	sX += kk
add.carry	000100	sX += sY + C
add.carry.im	000101	sX += kk + C
sub	000110	sX -= sY
sub.im	000111	sX -= kk
sub.carry	001000	sX -= sY - C
sub.carry.im	001001	sX -= kk - C
and	001010	sX &= sY
and.im	001011	sX &= kk
or	001100	sX |= sY
or.im	001101	sX |= kk
xor	001110	sX ^= sY
xor.im	001111	sX ^= kk
compare	010000	sX - sY
compare.im	010001	sX - kk
compare.carry	011110	sX - sY - C
compare.carry.im	011111	sX - kk - C
test	010010	sX & sY
test.im	010011	sX & kk
test.carry	101100	sX & sY (chain C, Z)
test.carry.im	101101	sX & kk (chain C, Z)
load	010100	sX = sY
load.im	010101	sX = kk
input	010110	sX = P[sY]
input.im	010111	sX = P[kk]
output	011000	P[sY] = sX
output.im	011001	P[kk] = sX
fetch	011010	sX = M[sY]
fetch.im	011011	sX = M[kk]
store	011100	M[sY] = sX
store.im	011101	M[kk] = sX
10....		

Instruction	Opcode	Description
rl	100000	
rr	100010	
sl0	100011	sX = (sX << 1)
sl1	100100	sX = (sX << 1) | 1
sla	100101	
slx	100110	
sr0	100111	sX = (sX >> 1)
sr1	101000	sX = (sX >> 1) | (1 << 7)
sra	101001	
srx	101010	
11....		
jump	110000	PC = aaa
jump.z	110001	if (Z) PC = aaa
jump.nz	110010	if (!Z) PC = aaa
jump.c	110011	if (C) PC = aaa
jump.nc	110100	if (!C) PC = aaa

4. Appendix

4.1. ALU Operations

Operation	Name	Description
ALU_ADD	Add	A + B
ALU_ADDCY	Add with Carry	A + B + Carry
ALU_SUB	Subtract	A - B
ALU_SUBCY	Subtract with Carry	A - B - Carry
ALU_AND	Bitwise AND	A AND B
ALU_OR	Bitwise OR	A OR B
ALU_XOR	Bitwise XOR	A XOR B
ALU_RL	Rotate Left	Rotate A left by 1 bit
ALU_RR	Rotate Right	Rotate A right by 1 bit
ALU_SL0	Shift Left (insert 0)	Shift A left, insert 0
ALU_SL1	Shift Left (insert 1)	Shift A left, insert 1
ALU_SLA	Shift Left Arithmetic	Shift A left, insert carry
ALU_SLX	Shift Left Extended	Shift A left, insert LSB
ALU_SR0	Shift Right (insert 0)	Shift A right, insert 0
ALU_SR1	Shift Right (insert 1)	Shift A right, insert 1
ALU_SRA	Shift Right Arithmetic	Shift A right, insert carry

Operation	Name	Description
`ALU_SRX`	Shift Right Extended	Shift A right, insert MSB (sign extension)
`ALU_TEST`	Test	A AND B, carry = odd parity
`ALU_COMPARECY`	Compare with Carry	A - B - Carry (flags only)
`ALU_TESTCY`	Test with Carry	A AND B, chain carry and zero
`ALU_PASS_B`	Pass B	Output B unchanged
`ALU_NOP`	No Operation	Output zero